Retrieval-Assisted Scalar Grounding for Natural-Language Optimization: A Diagnostic Study

Soroush Vahidi[1*]

[1*]Department of Computer Science, Ying Wu College of Computing, New Jersey Institute of Technology, University Heights, Newark, 07102-1982, NJ, USA.

Corresponding author(s). E-mail(s): sv96@njit.edu (ORCID: 0000-0003-1934-6282);

Abstract

Translating natural-language requirements into optimization models requires both selecting a mathematical structure and grounding its parameters in textual evidence. Strong retrieval or well-formed output alone cannot establish that these parameters express the intended problem. We investigate this distinction through a diagnostic evaluation that separates schema retrieval, scalar coverage and type checks, and numerical agreement across all required scalar parameters. On 331 NLP4LP queries and a fixed catalog of 335 schema documents, lexical and pretrained dense retrieval identify the target schema for 301 and 313 queries, respectively. However, ratio-aware lexical grounding passes the coarse readiness criterion on 265 queries (80.06%), while only seven (2.11%) meet the full stored-reference scalar-tolerance criterion. The evaluated pipeline uses reference-derived parameter inventories and query-reference scalar eligibility, so these results characterize assisted grounding rather than autonomous formulation. A separate study of Gemini, DeepSeek served through io.net, and Cohere covers 217 relationship judgments per model across 16 NL4Opt training problems. After historical recovery, 631 of 651 judgments are structurally valid; agreement on 197 complete cases yields Fleiss' $\boldsymbol{\kappa = 0.371}$, without establishing semantic accuracy. Twelve contrast pairs, validated by the author with language-model assistance, additionally illustrate distinctions that semantic evaluation must preserve, without supplying new provider-performance measurements. Together, the analyses show the need to evaluate retrieved structure, numerical completeness and semantic fidelity separately. The conclusions are limited to exposed catalogs, reference-assisted inputs and the available archived evidence.

Keywords: natural language processing, optimization modeling, knowledge representation, information retrieval, semantic grounding, intelligent information systems

Acknowledgements. The author is deeply grateful to his mother for her continuous emotional support. The author gratefully acknowledges Professor Ioannis Koutis of the New Jersey Institute of Technology (NJIT) for his academic guidance, encouragement, and continued support. The author also thanks NJIT for providing access to the Wulver high-performance computing cluster, which supported the computational research.

Special thanks are extended to Anders Borum, the developer of Secure ShellFish, for generously providing complimentary lifetime access to the application. Secure ShellFish facilitated secure remote access to computational resources and supported the research and development workflow.

The author also appreciates research computing support provided through the Cohere Labs Catalyst Grant Program, Google Cloud Research Credits Program, and io.net. This acknowledgment recognizes support to the author's broader research activities; use of these credits for the experiments reported here has not been established.

1 Introduction

Natural-language descriptions are an important source of data for optimization-based decision support. They specify what can be produced, which resources are available, how costs are incurred and which outcomes should be preferred. Turning those descriptions into a mathematical model requires more than recognizing relevant concepts or extracting numbers: each quantity must be assigned to the right entity, unit and mathematical role. A capacity and a cost can share the same numerical value but belong in different parts of a model. Likewise, changing "at most" to "at least" preserves the number while reversing the constraint. Such distinctions determine whether a model represents the decision problem that an analyst intended.

Research on natural-language optimization has approached this translation through entity extraction, structured representations and the generation of formulations or solver code [3, 16, 26, 27]. The need to connect textual evidence to a structured representation also appears in schema and value grounding for database queries [10]. Recent text-to-optimization research explicitly distinguishes selecting a model structure from binding its data [12]. The remaining evaluation question is therefore not simply whether grounding is difficult, but which measurements reveal its failures and which can conceal them. A retrieved template may be appropriate while its parameter assignments are wrong; a complete-looking mapping may satisfy type checks while misrepresenting a resource limit; and an executable program or a consensus label may still lack evidence of semantic correctness.

We study this question through *retrieval-assisted scalar grounding*. A retriever selects a schema document from a fixed catalog, and a deterministic procedure extracts

numerical mentions and assigns them to scalar parameter slots. The resulting partial mapping is an intermediate representation, not a complete optimization formulation. Lexical retrieval and a pretrained dense alternative make it possible to examine retrieval and grounding separately, while assignment controls examine how candidate evidence and search affect the result. These established components serve as inspectable instruments for evaluating the transition from textual data to a formal representation.

The evaluation setting is deliberately explicit about its assistance. The catalog is derived from the benchmark and contains all target schemas; it is not a test of generalization to unseen mathematical structures. The historical pipeline obtains parameter keys from reference-derived schema metadata, with a reference-record fallback, and filters participation using the query reference's numeric scalar keys before assignment. Although the numerical selector does not receive the reference magnitudes, the overall pipeline is not gold-free. Its measurements describe what happens when part of the required structure is already supplied, making it important to distinguish success at an intermediate check from agreement across the full reference scalar inventory.

Three questions organize the study. **RQ1:** How reliably can a relevant schema be retrieved in this known-catalog setting? **RQ2:** Why does successful retrieval or coarse readiness fail to ensure complete scalar agreement, and what do controlled assignment comparisons reveal about that gap? **RQ3:** What can structural validity, agreement and source-grounded relationship judgments establish about semantic interpretation? The third question separates observed model behavior from the reference evidence needed to assess correctness; it does not treat agreement as a substitute for that evidence.

The primary evaluation uses 331 queries from NLP4LP [2] and 335 schema documents. Term frequency–inverse document frequency (TF–IDF) retrieval identifies the target schema for 301/331 queries (90.94%), and pretrained BGE-M3 retrieval does so for 313/331 (94.56%). In the ratio-aware TF–IDF grounding configuration, 265/331 queries (80.06%) pass coarse readiness, but only 7/331 (2.11%) meet the stored-reference tolerance criterion for every required scalar. This separation is the central empirical finding: a favorable retrieval or readiness result does not certify a complete numerical mapping. Fixed-score assignment controls on a separate 49-query pilot further distinguish changes in slot assignments from aggregate whole-instance gains. Their scope is the retained candidates and scores, rather than an unrestricted test of better semantic representations. Imperfect references and missing original scalar mappings also limit interpretation of stored-reference agreement as source-level correctness.

A complementary historical study examines structured relationship judgments from Gemini, DeepSeek served through io.net, and Cohere. Each model judges the same 217 relationships drawn from 16 NL4Opt training problems. The recovered records support 631/651 structurally valid judgments and Fleiss' $\kappa = 0.371$ over 197 complete cases. These are validity and agreement measurements on relationships clustered within problems, not semantic-accuracy estimates or an independent model ranking. A separate common-18 formulation comparison provides limited execution and terminal-objective evidence under heterogeneous protocols. It cannot be combined with relationship labeling or scalar grounding into a single performance measure.

To make the semantic distinctions concrete, we additionally use a diagnostic set of 12 controlled pairs, comprising 24 samples across eight categories. Ten pairs change meaning and two preserve meaning through paraphrase. Their labels have been validated by the author through a documented review assisted by a language model; they are not independently blinded gold labels. The cases illustrate numerical bounds, units, roles, variable references, ratios, logical conditions and irrelevant information. They share generation lineage and recurring mathematical templates, and no provider pilot was executed on this set. Their contribution is to specify interpretable semantic probes, not to demonstrate new model-specific failures or explain causally the whole-instance grounding count.

The contributions are threefold:

1. An empirical decomposition of fixed-catalog retrieval, coverage/type readiness and agreement across all required scalar parameters, with explicit input assistance and denominators. It exposes the distance between selecting a suitable structure and instantiating its numerical content.
2. Controlled assignment and archived-measurement analyses that separate candidate, scoring and search effects from changes caused by evaluation procedures. These analyses delimit what the retained grounding and execution records can support.
3. A complementary examination of real-model relationship validity and agreement, together with author-validated semantic contrast cases. This distinguishes observable output conformity from the reference evidence needed for correctness claims.

The resulting contribution to data science is an evaluation methodology for assessing how textual evidence becomes a structured decision representation. It supports interpretable diagnosis and reproducible scrutiny of intermediate measurements, while retaining the limits imposed by benchmark exposure, reference assistance and incomplete archives. Section 2 situates the study; Sections 3 and 4 define the task and protocol. Section 5 presents the grounding diagnostics, Section 6 examines the separate large-language-model evidence, and Section 7 discusses implications and limitations.

2 Related Work

Recent advances in natural-language-to-optimization modeling reflect a shift from entity extraction to end-to-end generative systems, and increasingly to structured grounding and retrieval. We organize this literature around four primary axes that situate the present study.

2.1 End-to-End LLM Optimization-Model Generation

End-to-end systems use LLMs to generate formulations or solver code, extending earlier entity and meaning-representation tasks such as NL4Opt [27]. OptiMUS and OptLLM combine formulation, code generation and debugging [3, 42]. LLMOPT and ORLM study trained modeling systems [14, 15]; OptMATH develops synthetic optimization data [22]. SIRL and OR-R1 use solver-informed learning [8, 11], while

DeepOR studies a foundation model for optimization [36]. Agentic and experience-based approaches include OptimAI, AlphaOPT and ORThought [17, 32, 38]. Their formulation or execution outcomes differ from this paper's conditional scalar-assignment diagnostics.

2.2 Retrieval, Memory, and Solver-Feedback Systems

Retrieval and memory can augment generative modeling: OPT2CODE combines retrieval with code generation [4]; Autoformulation and uncertainty-aware approaches search over modeling decisions [6, 39]. Chain-of-Experts coordinates specialized agents [35], while dual-cluster and experience-replay systems retain modeling or correction knowledge [37, 43]. Solver-independent formulation and question partitioning broaden the task further [20, 23]. These approaches generally use retrieval or feedback within open-ended generation; our catalog supplies the structure on which scalar assignment is conditioned.

2.3 Binding and Structured Grounding

As benchmarks mature, researchers have exposed a steep performance drop when transitioning from simple puzzles to complex formulations [21]. This drop has motivated a critical distinction between a system's *modeling* ability (selecting mathematical structure) and its *binding* ability (grounding specific numeric parameters and data from text into that structure). Recent work by Gao et al. explicitly identifies data binding as a major limitation of generative text-to-optimization systems, reporting an effective binding limit in open-ended formulations [12]. Earlier component-level approaches such as NER4OPT [16] framed optimization modeling as an information-extraction problem over natural-language entities and quantities. Cross-domain text-to-SQL and schema matching provide related alignment settings [1, 31, 40]. Formulation-equivalence checking addresses a further distinction: a program can execute successfully without expressing the intended optimization problem [41].

Our study addresses a narrower but complementary setting to the binding challenges identified by Gao et al. [12]. We do not claim priority for the observation that binding is difficult. Their Text2Opt-Bench separates modeling from data binding as numerical scale grows. The July 2026 revision distinguishes oracle-fed BIND data externalization from a deployable trained binding specialist, and analyzes an untrained two-stage pipeline that fails to recover that benefit. Our narrower increment is the mismatch between assisted inventory/readiness gates and the all-key endpoint, together with frozen-objective and archival-measurement diagnostics. Our evidence concerns a different, already exposed benchmark and does not reproduce their scaling or intervention results. Rather than generating an open-ended formulation and then binding its data, we retrieve one schema from a fixed catalog and measure how retrieval errors alter the scalar slot inventory available to deterministic grounding—via oracle schema selection, StrictInstantiationReady, and an exhaustive residual-error decomposition.

Value grounding has related precedents outside optimization modeling. StruG learns column grounding, value grounding and column–value mapping for text-to-SQL [10]. Unit Dependency Graphs represent dependencies between quantities and units in

arithmetic word problems [29], while ILP-based semantic-role inference combines local evidence with linguistic constraints [25]. CQE represents quantities with units and associated concepts [5]. Execution-guided decoding rejects some faulty partial SQL programs [34]; successful execution still does not establish that an optimization model expresses its source problem. These connections motivate separate candidate, binding and consistency diagnostics; they do not validate the present heuristic or imply that it implements those systems. RAT-SQL explicitly learns schema relations and linking [33], whereas PICARD constrains generated sequences through incremental parsing [30]. These are precedents for structured alignment and output validity, not implementations or directly comparable baselines for our scalar-assignment task. In particular, a valid JSON judgment or executable program does not certify an occurrence–parameter relationship.

2.4 Position of the Present Work

The present study isolates fixed-catalog retrieval and scalar assignment. Its lexical path is inference-time LLM-free, but its evaluated inputs remain reference-assisted. Table S1 compares tasks rather than incomparable accuracy numbers; Section 5.3 provides the matched conditional comparison. Maximum-sum one-to-one assignment is classical [18]; the controlled decoder comparisons test its effect under frozen evidence, not a new matching algorithm.

3 Task Definition and Evaluation Assumptions

3.1 Problem Formulation

We study retrieval-assisted instantiation for natural-language optimization descriptions. Given a query, the system returns (i) a top-1 schema from a fixed catalog and (ii) scalar parameter assignments for that schema—an intermediate step before full model synthesis [2, 3, 27]. Schema retrieval ranks $\mathcal{S} = \{s_1, \ldots, s_M\}$ and returns

$$\hat{s} = \arg\max_{s \in \mathcal{S}} \text{score}(q, s). \tag{1}$$

Parameter instantiation then extracts numeric mentions and assigns them to scalar slots of $\hat{s}$ with deterministic rules. Incorrect retrieval can mis-specify the eligible slot set even when the query contains usable numeric evidence.

A *schema* here is the retrieved template together with its scalar parameter structure. The benchmarked setting does not synthesize a complete program, recover all variables/constraints, or reconstruct list-/vector-/dictionary-valued objects. Scope is restricted in three deliberate ways: (1) scalar numeric parameters only; (2) deterministic grounding (no generative LLM or learned extractor; no fine-tuning), with retrieval evaluated via lexical methods and pretrained BGE-M3; (3) schema-conditioned scalar recovery rather than end-to-end generation. Proxy checks and gold-assisted reference-program execution later provide limited downstream diagnostics without redefining this core task (Section 5.9).

Let $K_G(q)$ contain keys whose gold values are numeric scalars (integers or floats, excluding booleans), and let $P_{\text{all}}(\hat{s})$ contain *all* keys in the predicted schema's parameter-metadata dictionary, falling back to all keys in its reference parameter record when the parameters metadata is not a dictionary. This original construction does not first select entries by scalar shape. The benchmark constructs assignments on the eligible set

$$E(q, \hat{s}) = K_G(q) \cap P_{\text{all}}(\hat{s}), \tag{2}$$

Thus gold scalar status filters the slot inventory *before* assignment; gold keys affect both prediction construction and evaluation. Gold numerical magnitudes are not supplied to the mention-selection rule. Let $y_q : K_G(q) \to \mathbb{R}$ be the gold map and $\hat{y}_q$ the partial assignment on $A(q) \subseteq E(q, \hat{s})$; masked inputs may leave a raw unknown token rather than a numeric value. Retrieval quality is top-1 schema match; downstream quality is how well eligible scalars can be filled from text. As shown later, retrieval on NLP4LP is already strong, while residual difficulty concentrates in scalar grounding.

3.2 Evaluation Inputs and Reference Assistance

Reference access must be distinguished from reference scoring. The main four-arm and matched-baseline experiments are *gold-assisted*: the expected scalar count and slot identities depend on $K_G(q)$ before a value is chosen. Slot-name type heuristics consequently also operate on reference-derived names. The numeric selector does not receive the gold magnitudes, objective coefficients or constraint identities as separate targets; this narrower blindness does not make the complete pipeline autonomous.

The catalog itself is benchmark-derived. Its builder pairs each test description with the same record's parameterized description (falling back to its description and a metadata signature). The retained catalog contains 331 test-record documents plus four fillers; all 331 target IDs are present, and six query–target texts match exactly. This is a known-template identification setting, not a held-out template-generation test. Unsupervised fitting of an index is compatible with that setting, but does not establish independence from the evaluation source.

The later joint/factorial controls remove the query-specific gold-key filter, yet retain the selected reference template's slot names, shapes and descriptions, with types inferred heuristically from names. They remain *partially assisted*; available parameter relations also come from that template. Their results measure assignment conditional on supplied structure. By contrast, the 20-case execution check additionally retains reference code and unfilled scalar/structured values and is gold-assisted. Supplement, Section S3.3 identifies these boundaries and separates prediction reconstruction from aggregation. No autonomous formulation score, or adjustment estimating one from these conditional scores, is identifiable from the saved records.

3.3 Inference Dependencies and Practical Scope

The primary path uses a catalog-fitted TF–IDF representation, cosine ranking, numeric parsing and typed assignment. It calls no LLM and requires no task-specific neural training or LLM-generated relationship labels. Optional BGE-M3 retrieval does

use a pretrained neural encoder. External generation baselines, the auxiliary annotation audit, and research/writing assistance use LLMs separately; the research workflow is not wholly LLM-free.

With dependencies and reference metadata cached, the lexical path runs locally without model-service calls or per-token API charges. These are implementation properties, not measured comparative cost, privacy or reliability benefits. Deterministic replay additionally requires the original ordering/environment; reference-assisted inputs still prevent interpreting this path as autonomous deployment.

3.4 Metric Levels and Denominators

Schema R@1 is $N^{-1}\sum_{i=1}^{N} H_i$, where $H_i = \mathbb{1}[\hat{s}_i = s_i^*]$ and $N = 331$. For downstream metrics, write $E_i = E(q_i, \hat{s}_i)$ and let $A_i \subseteq E_i$ be the assigned slots. Let $T_i \subseteq A_i$ contain assignments accepted by the implemented coarse-type compatibility predicate. This is not literal type equality: an integer mention is accepted for a float slot, and integer or float mentions are accepted for currency slots, in addition to same-type matches. Expected slot types are inferred heuristically from names, not annotated semantic roles. Define

$$c_i = \begin{cases} |A_i|/|E_i|, & |E_i| > 0, \\ 0, & |E_i| = 0, \end{cases} \qquad \textit{Coverage} = \frac{1}{N}\sum_{i=1}^{N} c_i, \tag{3}$$

$$t_i = \begin{cases} |T_i|/|A_i|, & |A_i| > 0, \\ 0, & |A_i| = 0, \end{cases} \qquad \textit{TypeMatch} = \frac{1}{N}\sum_{i=1}^{N} t_i. \tag{4}$$

Both reported scores are *macro-averages over queries*, including zero-denominator queries as zero. They are not pooled slot ratios. Separate type-stratified diagnostic files pool slots by type and therefore use a different aggregation. An abstention leaves a slot unfilled, reducing coverage; TypeMatch conditions on assigned slots within each query, then averages across all queries. Missing retrieval yields an empty eligible set and zero coverage and type score. A masked mention can count as assigned while lacking a comparable numerical value; the implemented unknown mention type does not pass the coarse-type predicate.

For a schema-hit query, let $C_i \subseteq A_i$ contain slots with both a numeric prediction and numeric gold value. For each comparable slot,

$$e_{i,s} = \frac{|\hat{y}_{i,s} - y_{i,s}|}{\max(1.0, |y_{i,s}|)}. \tag{5}$$

A slot passes the inclusive tolerance rule if $e_{i,s} \leq 0.20$: absolute error at most 0.20 for $|y| < 1$, otherwise relative error at most 20%. No additional type gate is imposed on this numerical comparison. With $J = \{i : H_i = 1,\ |C_i| > 0\}$,

$$x_i = \frac{1}{|C_i|}\sum_{s \in C_i} \mathbb{1}[e_{i,s} \leq 0.20], \qquad \textit{Exact20 (on hits)} = \frac{1}{|J|}\sum_{i \in J} x_i. \tag{6}$$

The CSV field `exact20` stores x_i, a fraction of slots, not a whole-query indicator. It is blank on schema misses or when C_i is empty. Such queries are excluded, rather than zero-filled, in the reported Exact20 mean; an empty J is reported as unavailable. Main-table denominators are 291, 291, 303, and 320 *queries* for TF–IDF baseline, TF–IDF ratio-aware, BGE-M3, and Oracle. Missing numerical predictions do not count as errors in this conditional metric, so abstention can improve Exact20 while lowering coverage. Its method-dependent subsets do not support an unconditional accuracy ranking.

InstantiationReady retains its historical metric identifier for reproducibility and is described here as *coarse readiness*:

$$\mathit{InstantiationReady}_i = \mathbb{1}[c_i \geq 0.8 \ \wedge \ t_i \geq 0.8]. \tag{7}$$

The reported score is its mean over all N queries. It checks neither schema correctness nor numerical values, and can tolerate unfilled or incompatible slots. StrictInstantiationReady additionally requires $H_i = 1$; it still does not check values.

$$\mathit{StrictInstantiationReady}_i = \mathbb{1}[H_i = 1 \ \wedge \ c_i \geq 0.8 \ \wedge \ t_i \geq 0.8], \tag{8}$$

Empty eligible or assigned sets fail both readiness criteria. Thus an 80% coarse-readiness rate is not an 80% correct-instance rate.

4 Methods and Experimental Protocol

4.1 Retrieval-Based Schema Identification

The retriever ranks the 335 benchmark-derived schema descriptions and passes only its top-1 identifier downstream. This is known-template identification, not open-domain search or unseen-template induction. The selected template exposes parameter keys; Eq. (2) then applies the benchmark's gold scalar filter.

We evaluate BM25 [28], TF–IDF cosine retrieval [24], LSA [9], and pretrained dense BGE-M3 [7] (Table S2). Classical methods fit their unsupervised representations to the fixed catalog without query-label supervision; BGE-M3 is used off-the-shelf. No novel-schema generalization is evaluated. Equation (1) defines the selected schema; only that top-1 result is passed downstream. An *oracle* control substitutes the gold schema to expose grounding under perfect retrieval. A *random* reference is $1/335 \approx 0.0030$ for retrieval-only evaluation, or a uniformly chosen schema passed through the same grounding pipeline for downstream evaluation. Wrong retrieval can shrink or misalign the eligible slot set; strong retrieval supplies a more appropriate inventory for deterministic grounding.

4.2 Deterministic Parameter Instantiation

Given a query and predicted schema, the second stage produces slot–value assignments for eligible scalar parameters. It does not generate a complete model or recover variables, constraints, or structured objects. A fixed rule-based extractor recognizes

integers, decimals, written numbers, fraction words, percentages, currency-like forms, comma-separated magnitudes and the supported multiplicative ratio words; masked tokens in degraded settings are preserved as numerically unknown. Mentions receive a coarse type from {*percent*, *integer*, *currency*, *float*} via surface cues. Eligible slots are the intersection in Eq. (2); expected types are inferred from parameter names, and mention compatibility is defined below. This gold-assisted filtering is part of the evaluated protocol, rather than an independently inferred scalar inventory.

Typed greedy matching is the primary benchmark assignment rule. With eligible slots $\mathcal{P} = \{p_1, \ldots, p_m\}$ and an indexed collection of mentions $\mathcal{T} = \{t_1, \ldots, t_n\}$, where equal values at different source occurrences remain distinct mentions,

$$t_i^* = \arg \max_{t \in \mathcal{T} \setminus \mathcal{U}_{i-1}} \text{compat}(p_i, t), \tag{9}$$

where $\mathcal{U}_{i-1}$ are previously used mentions. Compatibility ranks coarse type preference, then absolute numeric magnitude, then raw surface text; remaining ties retain source order. Assigned mention indices are not reused, although equal values at other occurrences remain available. Slot order follows the recorded run's Python set iteration and is not portable without its interpreter/hash configuration. Section 5.3 compares this rule to other already-implemented same-task alternatives under fixed TF–IDF retrieval (typed greedy and constrained assignment use distinct token/mention constructors with related ratio-aware patterns; others retain intrinsic enriched extractors). Typed greedy remains strongest on InstantiationReady / StrictInstantiationReady; some alternatives raise conditional Exact20 by abstaining more. A selective schema-and-grounding reranking diagnostic (not a grounding-only comparison) showed an apparent InstantiationReady gain driven by wrong-schema artifact (Section 5.7). Algorithm 1 and Figure 1 summarize the common backbone.

Optional lexicon-based role cues for tie-breaking exist but are not part of the primary main-benchmark configurations and carry no formal correctness guarantee. A numeric-extraction ablation adds supported multiplicative ratio-word values (e.g., "twice" becomes 2 and "triple" becomes 3); experiments show gains in TypeMatch and InstantiationReady with retrieval unchanged.

4.3 Evaluation-Oriented Design Choices

Core metrics are solver-free. Reference-key filtering can let a wrong schema pass coverage/type gates on a small overlapping inventory; StrictInstantiationReady therefore adds schema match. Oracle selection is a control, not a formal upper bound on every metric. Neither readiness gate checks numerical values or complete-model correctness, and reference-program execution is evaluated separately.

4.4 Experimental Setup

We evaluate on NLP4LP [2, 3], a curated gated public benchmark of LP/MILP descriptions pairing each instance with a gold parameter record and reference solver code. Unless noted, results use the 331-query test split; each query has one gold schema and is ranked against the 335-document catalog (random top-1 chance $1/335 \approx$

Algorithm 1 Deterministic parameter instantiation: extract and type mentions; intersect predicted parameter keys with gold scalar keys; assign without token reuse under a deterministic compatibility rule.

Require: Query text q, predicted schema $\hat{s}$, gold scalar keys $K_G(q)$
Ensure: Slot–value mapping A

Extract numeric mentions $\mathcal{T}$ from q
Infer a coarse type for each mention in $\mathcal{T}$
Obtain parameter keys from schema $\hat{s}$
Set $\mathcal{P} = P_{\text{all}}(\hat{s}) \cap K_G(q)$; use the run's set iteration order
Infer an expected type for each slot in $\mathcal{P}$
Initialize assignments $A \leftarrow \emptyset$ and used-mention set $\mathcal{U} \leftarrow \emptyset$
for each slot $p_i \in \mathcal{P}$ **do**
    Define candidate set $\mathcal{C}_i \leftarrow \mathcal{T} \setminus \mathcal{U}$
    if $\mathcal{C}_i = \emptyset$ **then**
        continue
    end if
    Score mentions in $\mathcal{C}_i$ using a deterministic slot–mention compatibility rule
    Select the highest-ranked mention t_i^*
    Assign $A[p_i] \leftarrow \text{value}(t_i^*)$ if numeric; otherwise retain its raw unknown token
    Update $\mathcal{U} \leftarrow \mathcal{U} \cup \{t_i^*\}$
end for
return A

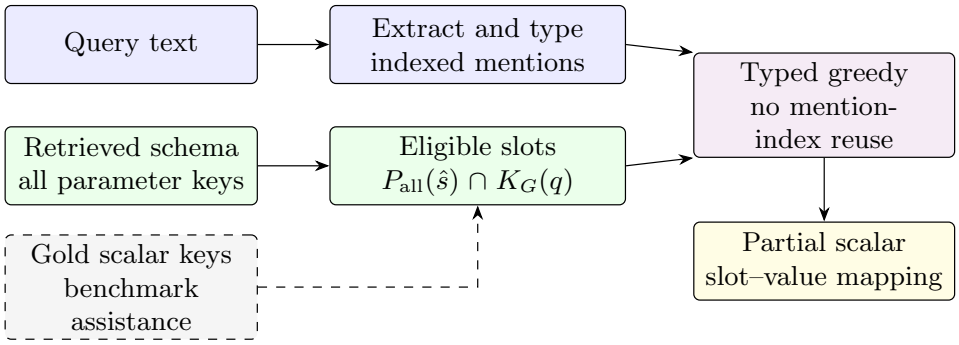


Fig. 1 Schema-conditioned deterministic parameter instantiation. The query supplies typed numeric mentions. All predicted-schema metadata keys, with reference-key fallback, are intersected with gold scalar keys to form participating slots; this benchmark assistance is explicit. Assignment consumes mention indices, not unique numerical values. The diagram describes the historical typed-greedy protocol; the later metadata-only factorial protocol uses a different inventory.

0.0030). These are historically designated test queries that have also been used repeatedly during development and retrospective analysis; the split name does not imply an untouched confirmatory evaluation. Three query variants probe robustness: *orig* (full text), *noisy* (controlled lexical corruption and numeric masking), and *short* (first sentence only).

Retrieval uses BM25, TF–IDF, LSA, and off-the-shelf BGE-M3 (no benchmark fine-tuning); only top-1 schemas enter grounding, with an oracle control for perfect

selection. Downstream grounding follows Section 4.2, restricted to scalar slots induced by the predicted schema. Reported configurations are: (i) TF–IDF typed-greedy with baseline extraction; (ii) TF–IDF typed-greedy with ratio-aware extraction; (iii) BGE-M3 with the same ratio-aware grounding; (iv) oracle typed-greedy. Configurations (ii) and (iii) use the same grounding implementation. However, their frozen assignments differ even on shared schema hits (Section 5.5); numerical differences cannot be attributed solely to retrieval.

Unless noted, retrieval and readiness metrics use the full denominator; Exact20 is the sole conditional exception. Paired significance uses the recorded resampling procedures with $B = 10{,}000$ and seed 42, plus exact McNemar tests where noted (Section 5.7). We also report overlap sanitization diagnostics, the numeric-extraction ablation (Section 5.5), and three restricted structural/solver studies (Section 5.9). Table S3 summarizes blocks. Classical retrieval and grounding are CPU-compatible. BGE-M3 reporting metadata specifies float32 L2-normalized 1024-dimensional embeddings and cosine similarity, but does not authenticate the execution device or complete environment. Data/software references include Hugging Face `datasets` [19] and SCIKIT-LEARN [24].

The preserved headline CSVs contain query-level evaluation metrics, not scalar maps. They match their first committed versions and support reconstruction of the reported summary statistics, but the original per-instance scalar assignments were not retained in the inspected archival outputs and cannot be independently reevaluated. Separate cached controls do preserve actual choices. Supplement, Section S3.3 distinguishes these evidence types; no newer predictions replace missing originals.

5 Diagnostic Results

5.1 Schema Retrieval Performance

Table 1 reports Schema R@1 across query variants for the 335-way catalog (random chance ≈ 0.0030). Within this fixed catalog—not as an open-domain claim—retrieval is already strong: BGE-M3 reaches 0.9456 on orig versus TF–IDF 0.9094 (15 BGE-only vs. 3 TF–IDF-only discordant wins; exact unadjusted $p = 0.0075$, Holm-adjusted $p = 0.0302$ in the family of 16). Noisy results remain close to orig; short truncations drop all methods but keep them well above chance (BGE-M3 0.8157). Mean R@1 computed from the unrounded rates across the three variants: BGE-M3 0.9013, TF–IDF 0.8640, BM25 0.8469, LSA 0.8328. We retain TF–IDF as the primary lexical downstream backbone and also evaluate BGE-M3 downstream to test whether residual grounding difficulty persists under stronger retrieval. These levels characterize identification in a benchmark-derived catalog. Together with the value-aware results below, they motivate examining the grounding stage; they do not establish performance on unseen templates.

Table 1 Schema retrieval accuracy across query variants, reported as *Schema R@1*. Each nonrandom value is reconstructed from saved top-1 schema IDs over all 331 test queries in the corresponding variant. The random baseline is the theoretical top-1 chance rate under uniform schema selection over the 335 candidate schemas, i.e., $1/335 \approx 0.0030$. Best result in each column is shown in bold.

Baseline	Orig	Noisy	Short	Avg.
Random	0.0030	0.0030	0.0030	0.0030
LSA	0.8459	0.8882	0.7644	0.8328
BM25	0.8822	0.8912	0.7674	0.8469
TF–IDF	0.9094	0.9033	0.7795	0.8640
BGE-M3 (dense)	**0.9456**	**0.9426**	**0.8157**	**0.9013**

Table 2 Main downstream diagnostics on the original queries, computed from frozen evaluation records. InstReady and Strict measure coarse readiness without checking numerical values. *Coverage*, *TypeMatch*, *InstantiationReady*, and *StrictInstantiationReady* use the full 331-query denominator. *Exact20* (on hits) is conditional on schema-hit cases with comparable predicted and gold numeric values. Best non-oracle result in each column is shown in bold.

Method	Coverage	TypeMatch	Exact20	InstReady	Strict
TFIDF-TG (baseline extr.)	0.8794	0.8515	0.2449	0.7764	0.7462
TFIDF-TG (ratio-aware extr.)	0.8886	0.8665	**0.2614**	0.8006	0.7704
BGE-M3 + ratio-aware	**0.9154**	**0.8946**	0.2436	**0.8248**	**0.8006**
Oracle-TG	0.9416	0.9230	0.2505	0.8489	0.8489

TFIDF-TG denotes typed-greedy grounding; Oracle-TG uses the reference schema. Exact20 macro-averages the per-query slot fractions in Eq. (6) over 291, 291, 303 and 320 comparable schema-hit queries, respectively; noncomparable cases are excluded. All cells aggregate saved query metrics, not retained scalar mappings.

5.2 Downstream Utility for Problem Instantiation

On the original queries, Table 2 reports macro-averaged Coverage and TypeMatch, conditional Exact20, and the two coarse-readiness rates. TF–IDF ratio-aware grounding reaches coarse readiness $265/331 = 0.8006$, while StrictInstantiationReady is $255/331 = 0.7704$; ten wrong-schema queries pass the ordinary thresholds. BGE-M3 and Oracle attain coarse readiness 0.8248 and 0.8489. These high rates coexist with low whole-query scalar correctness: 7/331 for TF–IDF ratio-aware, 3/331 for BGE-M3, and 8/331 for Oracle under the tolerance rule (Section 5.5). TF–IDF's larger conditional Exact20 (0.2614, versus 0.2436 for BGE-M3 and 0.2505 for Oracle) does not establish superior unconditional numerical accuracy, because evaluated subsets and frozen assignments differ.

Table 3 Matched same-task grounding baselines from saved metrics on the 331 original queries with TF–IDF retrieval held fixed. Queries and retrieval are fixed under the gold-assisted inventory rule; candidate construction, context, scoring, abstention and decoding can differ. Best InstantiationReady / StrictInstantiationReady values are shown in bold. Full_G requires the correct schema, all gold scalar keys, compatible types and complete stored-reference scalar tolerance; its denominator is 331 for every row.

Grounding method	Coverage	TypeMatch	Exact20	InstReady	Strict	Full_G
Typed greedy (reference)	0.8886	0.8665	0.2614	**0.8006**	**0.7704**	7/331
Constrained assignment	0.8531	0.8745	0.5772	0.7613	0.7311	40/331
Max-weight matching	0.8436	0.8740	0.5225	0.7432	0.7130	37/331
Optimization-role repair	0.8436	0.8724	0.5210	0.7372	0.7069	35/331
Search-structured grounding	0.8195	0.8795	0.6397	0.7039	0.6737	46/331
Semantic-IR repair	0.8131	0.8483	0.4949	0.7160	0.6949	9/331

Typed greedy and constrained assignment share ratio-aware patterns but use different token/mention constructors and context. Their comparison changes scoring, abstention and decoding together; it does not isolate global search. The other rows use intrinsic enriched extractors as part of their grounding definitions. *Exact20 (on hits)* denominators are 291 comparable schema-hit queries for all rows except semantic-IR repair (285), under the same exclude-noncomparable convention as Table 2.

5.3 Matched Same-Task Grounding Baselines

Under fixed TF–IDF retrieval, Table 3 compares mechanism-diverse grounding alternatives on the same 331 queries and catalog, using shared InstantiationReady and Exact20 definitions. The protocol records that the alternatives were selected before their matched scores were computed, with no additional tuning against the 331 labels for this comparison. Typed greedy and constrained assignment use related ratio-aware parsing logic but different token/mention construction and context; their scores and decoding also differ. Other rows use intrinsic enriched extractors required by their scoring definitions. Typed greedy remains best on InstantiationReady (0.8006) and StrictInstantiationReady (0.7704); all five alternatives have lower coarse readiness and Strict scores. A correction-only analysis of the saved paired labels, using the 16-comparison family defined in Section 5.7, gives adjusted $p \leq 0.0059$ for those ten readiness comparisons. Higher Exact20 for some alternatives (e.g., search-structured 0.6397) comes with lower Coverage, reflecting an abstention tradeoff. Coarse readiness alone does not determine which pipeline has better numerical correctness. The common all-key endpoint is 7/331, 40/331, 37/331, 35/331, 46/331 and 9/331 in the table's row order. Search-structured has the largest saved all-key count here; this descriptive ordering does not isolate a search effect or certify source semantics.

5.4 Controlled Grounding Diagnostics and Numerical Correctness

Coarse readiness and whole-query numerical correctness rank the existing pipelines differently. Saved records yield 40/331 full-gold-key scalar tolerance successes for constrained assignment versus 7/331 for typed greedy, with 36 gains and three losses. The other matched all-key counts are 37/331 for maximum-weight matching, 35/331 for role repair, 46/331 for search-structured and 9/331 for semantic-IR repair (Table 3).

The constrained eligible-only count is 41: query 102 covers only three of seven gold scalar keys and is excluded from the all-key count. Candidate construction, context, scoring, abstention and decoding differ together; both methods already prohibit mention reuse. Consequently, the gain cannot be attributed solely to global search.

A separate retrospective factorial study uses 49 primary queries and 277 gold slots, holding each candidate pool and score matrix fixed within decoder comparisons. The 64 collected cases additionally contain 12 development cases, one historical-overlap reconciliation case, and two reference-only cases. Inference uses predicted-schema scalar metadata without gold-key filtering, but still receives reference-derived slot structure. Under contextual scores and one-to-one assignment, exact slot counts increase from 106 to 136 for typed candidates and from 100 to 133 for rich candidates, but whole-query tolerance successes remain 5/49. These decoder contrasts do not survive the separately retained Holm family of 240 exploratory endpoint tests. With reusable mentions and additive scores, independent selection is already globally optimal.

The earlier joint-grounding pilot likewise finds 3/49 successes for both local and joint selection on the overlap-free slice. Of the original 50 cases, 32 have a criterion-correct mapping in the common pools, but only eight retain one in the beam's top ten and three select it. Exact diagnosis finds 23 strict objective-preference failures and six tied-score ambiguities among those 32. A subsequent exact value/type-distinct enumeration on the same 49 primary cases leaves retention at 9/49 and selected correctness at 3/49. More search or numerical diversity therefore produced no observed whole-instance improvement for these frozen objectives. These are conditional negative findings on a small pilot, not an equivalence result or evidence that better-supported constraints cannot help.

Appendix S3 retains the full factorial and pruning evidence, reference-convention corrections and reproduction map. All these comparisons are retrospective; the known query-291 overlap is excluded from new primary comparisons, but prior benchmark exposure remains.

5.5 Error Analysis and Ablation Studies

Whole-query scalar correctness is assessed by a mutually exclusive residual partition, evaluated in order: (1) wrong schema if $H_i = 0$; (2) incomplete coverage if $H_i = 1$ but $c_i < 1$; (3) type mismatch if $H_i = c_i = 1$ but $t_i < 1$; (4) value inaccurate if $H_i = c_i = t_i = 1$ but $x_i < 1$; (5) fully correct if $H_i = c_i = t_i = x_i = 1$. Here "fully correct" means *scalar correctness within the tolerance in Eq.* (5), not exact equality or correctness of a complete optimization model. Numerical evidence is required: missing Exact20 is never treated as fully correct. In these original-query records, every extracted mention is numeric; all eligible gold values are numeric, so full assignment makes every eligible slot comparable. Every observed fully correct case in the four main-table configurations also has key overlap 1, confirming coverage of all gold numeric scalar keys in those successes. The constrained eligible-only exception is reported separately in Section 5.4. Structured parameters and constraints remain outside this criterion.

For **TF–IDF ratio-aware typed greedy**, the five counts are 30, 31, 49, 214, and 7 out of 331 (Table 4). Thus $\mathbf{7/331 = 2.11\%}$ are fully correct, compared with $265/331 = 80.06\%$ passing coarse readiness. Cases labelled value-inaccurate against

Table 4 Mutually exclusive residual partition reconstructed from saved query metrics on all 331 original queries per configuration. Categories apply in the stated order; full coverage and types mean $c_i = t_i = 1$. Fully correct requires all eligible scalar values within the inclusive tolerance of Eq. (5), and does not certify complete model correctness. TF–IDF columns differ only in extraction; numerical differences involving BGE-M3 are descriptive (see text).

Category	TF–IDF base	TF–IDF ratio	BGE-M3	Oracle
Wrong schema	30	30	18	0
Incomplete coverage	48	31	33	35
Type mismatch	61	49	49	49
Value inaccurate	186	214	228	239
Fully correct (scalar tolerance)	6	7	3	8
Total	331	331	331	331

stored gold constitute $214/331 = 64.65\%$ of all queries, or $\mathbf{214/221 = 96.83}\%$ of the 221 queries passing schema, full coverage, and type gates. The latter is a conditional failure rate, not an unconditional error rate.

Saved query-level metric records support the same partition for BGE-M3 and Oracle, with fully correct counts $3/331 = 0.91\%$ and $8/331 = 2.42\%$. The value-inaccuracy rates conditional on passing the first three gates are $228/231 = 98.70\%$ and $239/247 = 96.76\%$, respectively. These conditional subsets differ by method. Moreover, TF–IDF ratio-aware and BGE-M3 share 298 schema hits but have different Exact20 values on 162 of them. The implementation constructs slot order using a Python set; TF–IDF records freeze `PYTHONHASHSEED=0`, but the BGE-M3 records do not document that seed. This is a reproducibility limitation and a possible source of assignment differences, not a verified causal explanation. These are descriptive results for frozen configurations, not a controlled estimate of retrieval's effect on numerical correctness. No historical predictions were regenerated or substituted.

The ratio-word extraction ablation in Table 5 raises Coverage from 0.8794 to 0.8886, TypeMatch from 0.8515 to 0.8665, InstantiationReady from 0.7764 to 0.8006, and Strict from 0.7462 to 0.7704 (8 strict gains, 0 losses; unadjusted McNemar $p = 0.008$). Retrieval is unchanged. The patch improves these coarse diagnostics, but the value-aware partition shows that most numerical errors remain unresolved. The frozen query-level records do not identify semantic subcauses such as totals versus unit coefficients.

5.6 Challenges in Numerical Mention Grounding

The stored-value failures motivate an occurrence-level relationship audit; they do not by themselves identify semantic error causes. A mention has a source occurrence, not just a magnitude: for the auxiliary relation audit, write $m = (a, b, v)$ for its half-open character interval and normalized value, and $r = (q, m, p)$ for the proposed relation to parameter p. This diagnostic representation does not imply that the historical typed-greedy outputs retained character-aligned assignment traces. Two occurrences with the same v can refer to different resources; one occurrence may also legitimately support

Table 5 Numeric-extraction ablation on the original queries, comparing the baseline and ratio-aware extraction configurations of the TF–IDF typed-greedy method. The ratio-aware configuration corrects multiplicative ratio-word numeric expressions. The strict coarse-readiness count (of 331) rises from 247 to 255; Δ is ratio-aware minus baseline; see Section 5.7 for the significance test.

Metric	Baseline	Ratio-aware	Δ
Coverage	0.8794	0.8886	+0.0091
TypeMatch	0.8515	0.8665	+0.0150
Exact20 (on hits)	0.2449	0.2614	+0.0165
InstantiationReady	0.7764	0.8006	+0.0242
StrictInstantiationReady	0.7462	0.7704	+0.0242

several declarations. No-reuse assignment is therefore an experimental restriction, not a general law of optimization modeling. Implicit objective or identity terms introduce a different problem: there may be no lexical number to align. Their absence cannot be converted automatically into a verified null relationship.

Historical weak-label learning used numerical equality to choose a first matching mention. Such targets do not distinguish repeated values, resource roles or incomplete formulations. Earlier neural and classifier-plus-assignment runs are documented, but their supervision and evaluation do not support a clean conclusion that learning cannot improve grounding. The saved P0 comparison shows no coarse-readiness gain from its exact-assignment condition; missing prediction traces and evaluation-derived splits prevent a broader semantic interpretation. Supplement, Section S3.5 distinguishes completed negative results, inconclusive runs and blocked proposals.

5.7 Statistical Significance and Lexical-Overlap Robustness

All inferential analyses here are retrospective. Query identities are paired across methods; the correction-only reanalysis reuses the existing 327 delexicalized text groups for all 331 original queries. It reports marginal paired group-percentile bootstrap intervals (B = 10,000, NumPy seed 42), exact groupwise method-swap tests on integer count differences, and Holm adjustment over the 16 reported retrieval/readiness contrasts [13]. That family contains five main-table contrasts, the BGE–TF–IDF retrieval contrast, and ten ordinary/Strict contrasts against the five grounding alternatives. The factorial's separate 240-endpoint family remains unchanged. Grouping is heuristic and does not remove all dependence or prior benchmark exposure.

Table 6 replaces the legacy significance summaries using the same saved observations. The earlier uncentered bootstrap sign-tail values were not null-calibrated tests; the BGE intervals were shifted randomization quantiles rather than sampling confidence intervals. Floating-point ties also affected those BGE p-values. Historical statistic files are retained in the correction record. The ratio-aware extraction gain remains $8/331 = 0.0242$; its exact paired/group test is $p = 0.0078125$ and the adjusted value is 0.0302. These tests concern coverage/type diagnostics, not numerical

Table 6 Correction-only paired reanalysis of the saved 331-query gate records (327 text groups). Difference is the first configuration minus the second; CI is a marginal 95% paired group-bootstrap interval. Exact p-values use groupwise method swaps; the final column adjusts the stated family of 16 tests. All comparisons are exploratory.

Comparison	Diff	95% CI	p	Holm p
TFIDF ratio – Oracle (coarse)	-0.0483	$[-0.0751, -0.0240]$	< 0.001	0.0028
TFIDF ratio – Oracle (Strict)	-0.0785	$[-0.1108, -0.0486]$	< 0.001	< 0.001
TFIDF ratio – baseline (Strict)	$+0.0242$	$[0.0091, 0.0423]$	0.0078	0.0302
BGE-M3 – TFIDF ratio (coarse)	$+0.0242$	$[0.0030, 0.0455]$	0.0386	0.0386
BGE-M3 – TFIDF ratio (Strict)	$+0.0302$	$[0.0091, 0.0534]$	0.0129	0.0302

Table 7 Sensitivity check: canonical *InstantiationReady* versus the stricter *StrictInstantiationReady* (Eq. (8)), all 331 original queries, computed from saved per-query metric records. Δ is standard minus strict; n_{differ} counts queries whose readiness label changes.

Method	InstReady	StrictInstReady	Δ	n_{differ}
TFIDF-TG (baseline)	0.7764	0.7462	0.0302	10
TFIDF-TG (ratio-aware)	0.8006	0.7704	0.0302	10
Oracle-TG	0.8489	0.8489	0.0000	0

correctness. The historical sanitized-retrieval aggregates (Table S4) report preserved or improved rates after stripping numbers and/or stopwords. Without the original per-condition predictions, this is limited support for robustness to that transformation, not independent confirmation of its mechanism. The diagnostic uses a closely matched index (331 documents excluding four filler templates; no `doc_id` tokens), so TF–IDF/BM25 baseline cells differ slightly from Table 1 while LSA matches exactly (0.8459). A Jaccard-overlap stratification places 327/331 queries in a high-overlap bucket (TF–IDF R@1 0.9174); the medium-overlap bucket has only 4 queries and is reported only as a qualitative caveat.

To check whether wrong schemas can inflate ordinary InstantiationReady via smaller eligible-slot sets, a repository diagnostic of selective schema reranking raised ordinary readiness from 257/331 to 265/331, but 6 of 8 newly ready queries used an incorrect schema. We therefore use the schema-gated readiness measure as a robustness check (Table 7). The gate removes 10 TF–IDF queries in both extraction configurations and widens the descriptive TF–IDF–Oracle readiness gap from 0.0483 to 0.0785. The corrected ordinary/Strict adjusted p-values are 0.0028 and < 0.001, respectively; neither metric tests scalar values.

5.8 Computational Complexity and Runtime

The recorded lexical backbone has a small computational footprint on this catalog. TF–IDF fits a sparse vectorizer once over the $M = 335$ schema documents and, for each query, transforms the query and scores it against the full catalog matrix

by sparse cosine similarity (SCIKIT-LEARN); with fixed small M, the lexical query path consists of sparse vector operations and does not execute a neural encoder. Rule-based extraction is linear in query length. Typed-greedy assignment scans candidates in $O(|\mathcal{P}| \cdot |\mathcal{T}|)$ comparisons in the (typically small) numbers of eligible slots and mentions. A historical timing summary for TF–IDF + ratio-aware typed greedy (331 queries, `PYTHONHASHSEED=0`) records wall time 1.09 s (3.29 ms/query) with peak RSS $\approx$ 198 MB. The recorded scope excludes BGE-M3 embedding load/compute and remote LLM baselines; the single retained timing summary is contextual, not a reproducible timing distribution or cross-method efficiency benchmark.

5.9 Structural and Solver-Backed Validation

Core metrics are solver-free. Three historical restricted studies use a different grounding implementation: optimization-role relation repair, rather than the headline typed-greedy configuration. The 60-case study takes the first original queries with 2–8 gold scalar parameters; the 269-case study selects instances with reference solver code. Their saved aggregate rates are retained, but their names must not be interpreted as stronger validation than the code performs.

The legacy field `structural_valid` is a *coverage/type proxy*: it is true when the predicted eligible inventory is empty, or when coverage and type match are both at least 0.8. Unlike main-benchmark readiness, this proxy treats an empty inventory as passing. It does not inspect linearity, decision variables, objective correctness or constraint well-formedness. The field `parseable_model_artifact` checks only that objective and constraint metadata are nonempty. `instantiation_complete` combines that presence check, full predicted-scalar coverage, and absence of expected parameter names in the rendered description; it does not require compatible types or validate numerical values. These subset inventories use numeric status in the predicted schema's parameter record, not the main benchmark's intersection with the query's gold keys.

The 20-case solver study is a separate, compatibility-filtered execution check. Its executable flag means that reference code finishes without an exception and leaves a model object. The two legacy success/feasibility flags both test the shim's status 2, set after a successful SciPy optimization with a returned solution; they are not independent outcomes. The objective-produced flag checks for a non-null objective value. Predicted-template parameter records initialize execution: unfilled scalars and structured data retain reference values, and filled parameters originally typed as integers are rounded. Thus these are gold-assisted executions of reference programs, not complete programs generated from the query alone.

Coverage/type proxy subset (60 instances). Table S5 retains the historical schema-hit, coverage/type-proxy and template-completion rates. Neither proxy tests numerical or formulation correctness.

Extended proxy subset (269 instances). On a larger subset with available reference solver code, the historical report records for TF–IDF, BM25, and Oracle schema-hit rates of 0.9368, 0.9257, and 1.0000; coverage/type-proxy rates of 0.8141, 0.8104, and 0.8253; and template-completion rates of 0.6654, 0.6580, and 0.6840. Solver execution was not evaluated here because the reference path required Gurobi support

Table 8 Offline reaggregation of archived common-18 records. Every system attempted generation on the same 18 cases. Parse denotes a retained AMPL parse or Python code extraction; Run denotes successful process/solve completion. Obj. counts valid numerical text extractions among successful runs, and Ref. counts available reference objectives among those extractions. Agreement is conditional on classifiable executed reference comparisons. W/H means withheld for insufficient precision; N/A means an unavailable execution environment.

System	Parse /18	Run /18	Obj./run	Ref./obj.	Agree./comp.
PaMOP reconstruction	13/18	13/18	13/13	11/13	W/H
OptMATH adaptation	18/18	15/18	15/15	15/15	11/15
ORLM adaptation	18/18	N/A	N/A	N/A	N/A
GPT-5.4 zero-shot	18/18	16/18	16/16	16/16	12/16

unavailable in the evaluation environment; executable conclusions are reserved for the HiGHS-compatible subset below.

Restricted solver-backed subset (20 instances). The historical script selects the first 20 reference programs passing a literal substring compatibility filter for the shim. It constructs one subset before either condition, but the original ID manifest is unavailable, preventing independent membership or paired-outcome checks. Table S6 retains its reported rates. The 0.80 versus 0.75 solver-success gap is one descriptive case, not a tested improvement. The report attributes non-executions to two shim `IndexError`s among 40 method–instance evaluations; original logs are unavailable.

The extended report records 69 missing-slot, 82 type-mismatch and 267 template-incompletion events across three baselines (807 method–instance evaluations). These categories can overlap and are not a partition of 269 unique queries. The completed archival investigation found no original subset manifests or outcome files. Case examples in the 60/269 reports establish some shared membership, not full samples; these studies must not be treated as disjoint. The complete rates remain aggregate-supported. The main numerical conclusions do not rely on these proxy checks.

6 LLM-Based Comparative Evidence

6.1 Archived Formulations on a Shared Subset

The scalar-grounding system emits a partial mapping rather than a complete program. Its readiness and scalar metrics are therefore outside the formulation comparison in Table 8; Supplementary Table S11 describes the different tasks and implementations. These are 18 historically selected problems, not an independent benchmark or an equal-budget model contest.

6.2 Objective Extraction, Precision and Failure Accounting

The fixed selection precedes the comparison outcomes and covers the same IDs in every row. Zero-shot GPT-5.4 uses the Azure OpenAI deployment returning `gpt-5.`

4-2026-03-05, one generation at temperature 0, and direct Gurobi code. The project-maintained PaMOP reconstruction uses the same snapshot at temperature 0.2 with decomposition and at most five correction iterations; its actual executable path is AMPL/Gurobi, including one AMPL presolve-only success. The project-maintained OptMATH adaptation uses the released Qwen2.5-7B checkpoint at temperature 0.8, and the ORLM adaptation uses its released LLaMA-3-8B checkpoint with greedy decoding. Different prompts, framing, weights, correction budgets and solver interfaces preclude a model or provider ranking.

The original Python execution harness selected the first objective-like string, which could be a heuristic incumbent. The correction instead requires a terminal optimal solver summary and checks it against the final standalone objective printed by the generated program. The original generated code, stdout and reference objectives are preserved; no program or model is rerun. Nine stored objective fields differ from these terminal values, and seven agreement labels change. All 31 successful Python runs have a valid terminal/printed objective pair. The corrected agreement counts are 11/15 for adapted OptMATH and 12/16 for zero-shot GPT-5.4, with four disagreements in each conditional set.

For continuity the reported Python proxy retains its original predicate. Let $a = \mathrm{round}_{\mathrm{even}}(\hat{z})$ and $b = \mathrm{round}_{\mathrm{even}}(z^*)$, using Python's float conversion and nearest-integer ties-to-even behavior. Agreement is

$$\mathbb{1}\left[\begin{cases} |a-b| \le 0.05, & b = 0, \\ |a-b|/|b| \le 0.05, & b \neq 0. \end{cases}\right]. \tag{10}$$

Full available numerical tokens are retained before that explicit comparison step. A separately recorded unrounded sensitivity uses absolute 0.05 at a zero reference and relative 0.05 otherwise; it gives identical classifications on these 31 cases. This agreement does not validate rounding for other objectives or make the systems' protocols identical.

PaMOP's historical absolute-10^{-6} objective comparison has a different precision problem. Common-18 case 254 displays 16666.7 although its solver summary prints 16666.66667 against reference 16666.666666666664. Even the latter text cannot certify that tolerance. More generally, the retained displayed equality does not establish an unrounded return at 10^{-6}. We therefore withhold its aggregate objective-agreement score. Among 11 executed cases with numerical references, two discrepancies exceed a conservative last-displayed-unit uncertainty bound and nine remain precision-unresolved; two other executed cases have no reference objective. An algebraic witness for case 254 is documented separately and is not substituted as a recovered numerical solver result. No revised exact-agreement fraction is inferred.

Five PaMOP cases finish with parse failure; nine of its 13 successful cases execute without correction and four after correction. Adapted OptMATH has three unsuccessful processes, and zero-shot has two. Four of these five Python failures (cases 219 and 237 for both systems) invoke Gurobi, terminate infeasible or infeasible-or-unbounded, and then fail accessing an objective; OptMATH case 232 has a distinct nonlinear-constraint API construction error. These stages are not interchangeable semantic

diagnoses. All successful runs report optimal/solved generated-model outcomes, with conditional feasibility counts 13/13, 15/15 and 16/16 for PaMOP, OptMATH and zero-shot, respectively. This does not show that the generated models express the source problems.

ORLM has code extracted on all 18 cases but no execution because `coptpy` is unavailable. Its execution and objective columns remain N/A. The supplement gives each case's outcome, source/configuration identities and precision disposition. Missing rendered historical prompts and incompletely authenticated environments still limit exact generation replay. Objective agreement and successful execution do not establish formulation equivalence; the core scalar-grounding method remains outside these columns.

6.3 Real-Model Judgments of Occurrence–Parameter Relationships

Retrieving a suitable schema leaves open which source occurrence expresses each parameter. We therefore examine whether actual LLM services give complete, structurally grounded and mutually consistent judgments on proposed occurrence–declaration relationships. This is a separate annotation/QC question from both the 331-query scalar-grounding study and the common-18 formulation comparison. It tests neither the autonomous correctness of the retrieval system nor accuracy against human labels.

The frozen October 27, 2026 records contain 217 distinct relationship items from 16 NL4Opt training problems [27], drawn as a category-covering, high-risk convenience packet from a previously inspected 100-problem review frame. They are not 217 independent problems, repeated independent trials, or a random corpus sample. Multiple relationships share problem text, numerical occurrences and declarations. Each service originally received 33 batches of up to eight targets, with complete source text, supplied variables, objective and constraints; peer judgments were withheld. The four nominal decisions are `CORRECT`, `INCORRECT`, `AMBIGUOUS` and `CANNOT_DETERMINE`. These are model assertions, not verified truth labels. Each object must also supply an explanation, literal source quotations, valid source identifiers, omission notes, a shared-mention flag and uncertainty.

The requested models are `gemini-3.1-pro-preview` through Google Vertex AI, `deepseek-ai/DeepSeek-V4-Pro-0813` through io.net io.intelligence (recorded FP8), and `command-a-reasoning-08-2025` through Cohere's direct v2 API. DeepSeek here names the served model, not a direct DeepSeek API. All use temperature 0 and a 12,000-output-token cap; Gemini uses LOW thinking, Cohere a 256-token thinking budget, and io.net its default reasoning behavior. Sampling/analysis seed 20261027 is not an inference seed. The identifiers authenticate the requested or echoed service names, not underlying weight checksums or every server default.

Strict output-contract checks require exact source quotations and membership in supplied variable/formulation identifiers, in addition to JSON structure. Recovery v2.1 preserves all 500 originally valid objects, extracts only unique complete envelopes, maps only explicitly documented source aliases and losslessly serializes structured omission notes. Of 92 model-item targets still invalid after offline repair, historical

Table 9 Structural validity on the same 217 relationship targets per service. Final counts combine original-valid objects, deterministic offline repairs and historically generated valid replacements under a revised format. They are not single-pass or equal-treatment accuracy estimates. No new calls were made for this revision.

Service/model family	Original valid	Offline repairs	Valid replacements	Final valid
Vertex AI/Gemini	215	2	0	217/217
io.net/DeepSeek	180	33	4	217/217
Cohere/Command A	105	24	68	197/217
Total	500	59	72	631/651

smaller-batch requests with explicit identifier/quotation instructions yield 72 valid replacements. Cohere additionally uses a native JSON Schema with identifier enums; an initial rejected schema and a corrected transport are retained in the lineage. These changes affect comparability. The remaining 20 invalid Cohere objects all contain nonliteral evidence quotations; invalidity does not prove a mathematically wrong decision.

Among jointly valid pairs, nominal agreement is Gemini–DeepSeek 213/217 (98.16%), Gemini–Cohere 123/197 (62.44%), and DeepSeek–Cohere 120/197 (60.91%). The complete three-model denominator is 197, comprising 119 determinate unanimous cases, 77 disagreements and one unresolved case. The remaining 20 of 217 are invalid/incomplete. The unresolved case has three `CORRECT` labels but no verified lexical endpoint, so raw label unanimity is 120/197, while determinate unanimity is 119/197. Neither is semantic accuracy.

Fleiss' nominal κ on the 197 complete cases is 0.370807 (observed pair agreement 0.739425, chance term 0.585858), using all four decision categories and excluding invalid objects rather than coding them as an opinion. The complete-case marginal counts are 420 `CORRECT`, 168 `INCORRECT`, two `CANNOT_DETERMINE` and one `AMBIGUOUS`, totaling 591 judgments. Original complete cases number only 89, with $\kappa = 0.501643$; those same 89 cases retain identical labels after recovery. The change in aggregate κ reflects expanded complete-case membership, not measured loss or gain in correctness. We report no population confidence interval, significance claim or model ranking for these clustered, selected items.

Verifiable defects include a singular variable alias rejected against a plural source identifier, prose/structured-note output that violates the schema, and quotations that paraphrase rather than copy the source. Valid decisions also differ on source-occurrence binding: Supplement, Section S6.1 gives identified witnesses and the per-problem denominator ledger. These examples explain protocol failures or conflicting model assertions, not independently adjudicated semantic error frequencies. No human-verified reference judgments exist for the 217 targets, and no consensus is approved as training gold. The study complements the retrieval/readiness gap: a retrieved template and a plausible structured judgment can both look usable while correct source binding remains unverified. Agreement cannot validate the retrieval pipeline's autonomous performance.

7 External Validation, Discussion, and Limitations

7.1 External Component-Level Robustness Validation

As a component-level check, we evaluated numeric extraction on a fixed 1,000-instance OptMATH-Train sample [22] (no fixed schema catalog, so extraction only). Pooled raw AST code-value overlap recall is 0.7501; pooled automatic text-support recall is 0.8118 (complete-instance rate 0.5910). A secondary rule-based classifier on 150 instances (seed 20260815) estimates that 7.28% of its inspected AST-value denominator are implementation, loop or solver constants. The later verification summary reports audit-calibrated text-supported recall as the pooled (micro) ratio $23{,}241/28{,}799 = 0.8070$ (95% instance-bootstrap CI $[0.7842, 0.8283]$, $B = 10{,}000$, seed 42, over the 1,000 sampled instances). Its corrected per-instance counts are unavailable for independent rebootstrap of that historical CI. The reported mean of per-instance ratios (macro recall) is 0.8135. The stored verification JSON reverses the names of these two aggregates; the values are preserved here with their implemented definitions. The 28,799 values are not independent samples, and occurrence-aware recall is not identifiable from these set-based artifacts. Two independently coded classifiers agree on 98.04% of 4,801 literals in the 150-instance sample—automated cross-validation, not human gold labels. This is limited evidence that extraction transfers beyond NLP4LP; it does not establish full-framework generalization.

7.2 Limitations and Threats to Validity

Conditional task and imperfect references. Main grounding uses gold scalar-key eligibility; later controls still receive reference template metadata. Neither demonstrates autonomous inventory discovery or complete formulation generation. Readiness omits value checks, while the full scalar criterion omits structured objects and model equivalence. Metadata may omit keys (query 102 exposes 3 of 7; query 293 has no eligible slots); successful cases in each of the four headline configurations cover all gold scalar keys, and the constrained eligible-only exception is disclosed separately. Stored references are not uniformly source-correct. The separate overlay in Supplement, Section S3.2 is not maintainer-certified gold, and absolute 0.20 tolerance for fractions can accept materially different proportions.

Prediction retention and replay. The inspected historical headline writers saved 13 metric columns, not scalar maps. The archival investigation authenticated the four metric CSVs, but found no matching assignment exports; deletion is not established. Stage-A, later cached choices and unverified archival candidates cannot replace them. Missing values and slot order prevent corrected-reference headline evaluation. BGE-M3's hash seed and complete execution environment are unrecorded; reporting metadata is not a runtime receipt. The three aggregate-supported tables and the extended subset study lack original per-condition or membership/outcome records (Supplement, Section S3.3). Gated dataset access also limits external reproduction.

Dependence and statistical scope. The main experiments reuse one benchmark and a catalog constructed from its source records. Sanitization does not establish template independence; only four queries occupy the medium-overlap bucket. Grouping and

the exclusion of query 291 address known text dependence but do not certify a fresh test set, independent templates or generalization across domains/languages. Earlier learned splits contain confirmed group crossings. Paired and factorial analyses are exploratory; multiplicity adjustments do not remove prior benchmark exposure. The selected 60/269/20 and common-18 studies support neither population-wide inference nor a harmonized end-to-end comparison.

Semantic supervision. No formal independently accepted human reference set exists. The selected NL4Opt relationship audit in Section 6.3 measures output structure and inter-model agreement, not semantic accuracy. Its recovery stages use different output-format instructions and incomplete-case selection changes the agreement population. It neither evaluates the 331-query pipeline nor supplies accepted training labels. Equality-derived weak labels, valid JSON, LLM consensus and source-semantic correctness are distinct. The targeted NL4Opt packet cannot estimate corpus-wide accuracy or exhaustive relationship recall, and its 713-problem source audit is not 713 certified independent training examples. Quarantine is precautionary; unflagged problems remain uncertified. Heuristic type/role cues have no correctness guarantee, and the residual partition cannot establish frequencies of particular semantic mechanisms.

Operational claims. API-free lexical processing is an implementation property; comparative monetary cost, privacy, reliability and usability were not measured. BGE-M3 uses neural inference. The historical single-run timing and extraction-only OptMATH check do not establish production performance or full-framework transfer. No deployed-system or user study was conducted.

7.3 Future Research Directions

A possible future prerequisite is independent human validation of occurrence–parameter relations under a frozen rubric and sampling frame. A separate first-round protocol prepares 36 probability-sampled and 12 targeted relationships, with two overlaps (46 distinct cases across 16 problems); it has not yet produced formal reference labels. Its inferential scope is the selected 217-item frame, not the whole NL4Opt corpus. Source-reference and template-independence review must accompany any future use as supervision. This addresses a measured evidentiary gap before another scorer or encoder comparison is justified. Structured parameters and broader solver-backed validation remain outside the present claims. No new experiment or training run was performed for this revision.

7.4 Closed Feasibility and Identifiability Audits

Three closed audits clarify the information boundary without adding predictions. G1 found no validated autonomous slot-discovery/eligibility path in the current system. G2 found that the native gold operation selects participating slots, so removing a supposed numerical-candidate filter at fixed required slots is either vacuous or changes the task. G3 instead compared participation sets using saved TF–IDF schema IDs and metadata scalars with `shape=[]`. Sets are equal in 301/331 cases and differ in 30/331, adding 134 slot occurrences; 172/1,847 reference scalar occurrences are absent from

predicted scalar inventories. Of the changed cases, 29 miss required reference keys under both conditions and one has no numeric scalar reference. Thus no informative whole-instance contrast is identified by those records. These are static methodological findings, not measured performance differences or tests of equivalence. Strict scalar metadata also changes the original all-key/fallback construction in seven cases, so the two inventory definitions cannot be silently substituted. Detailed set accounting is in the supplement. No further experiment was launched.

8 Conclusion

Retrieval accuracy, coverage/type readiness, numerical slot agreement and complete scalar agreement answer different questions. On the historical TF–IDF ratio-aware configuration, 265/331 queries (80.06%) pass coarse readiness, 255/331 pass its schema-gated version, and only 7/331 (2.11%) satisfy the full stored-reference scalar-tolerance criterion. All of these results condition on reference-derived participation and benchmark templates; none measures autonomous optimization.

Fixed-score assignment and exact-search diagnostics constrain claims about stronger decoding on the retained pilot, without establishing general equivalence. The corrected common-18 records support conditional terminal-objective proxy agreement of 11/15 for adapted OptMATH and 12/16 for zero-shot GPT-5.4. The PaMOP objective aggregate is withheld because archived precision is insufficient, while ORLM execution remains unavailable. These are evidence-qualified outcomes of heterogeneous formulation systems, not a capability ranking. The separate Gemini–DeepSeek–Cohere relationship audit exposes a complementary supervision boundary: 631/651 final objects pass the output contract, yet complete-case agreement ($\kappa = 0.371$ on 197 relationships) supplies no human semantic truth. Its selected, clustered frame and targeted recovery preclude provider rankings or general accuracy claims. The contribution is an evidence-qualified diagnostic and reproducibility account, not a new retrieval/matching algorithm, a provider ranking, or a claim that numerical binding difficulty is newly discovered.

Declarations

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. The external large-language-model baseline experiments used Microsoft Azure OpenAI services; associated usage was supported by the standard Azure for Students educational credit available to the author. NJIT/Wulver infrastructure access and complimentary Secure ShellFish access were institutional and in-kind assistance. The archived relationship-labeling runs identify providers but do not establish which research-credit awards paid for their use.

Competing interests. The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethics approval and consent to participate. Not applicable; this study did not involve human participants, human data, or animals. No formal human-annotation

study contributes reference labels; the future validation protocol has not been carried out.

Author contributions. Soroush Vahidi: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization. This single-author paper uses the standard CRediT taxonomy to describe the contributions of the sole author.

Data availability. Historical derived metrics are public at https://huggingface.co/datasets/SoroushVahidi/nl-optimization-instantiation-metrics, under their existing CC BY-NC-SA 4.0 license. NLP4LP is obtained separately from `udell-lab/NLP4LP`; its current public card identifies CC BY-NC 4.0 and gated access. We provide upstream access instructions rather than redistribute that local cache. OptMATH raw data have an unresolved dataset license and are likewise obtained upstream. The MIT-licensed NL4Opt source subset, relationship prompts, per-case judgments, validity calculations and recovery lineage are prepared in a separate local release candidate, with account identifiers removed from copies. This M5 candidate is prepared for author inspection and has not been uploaded. Supplement, Section S3.3 distinguishes directly verifiable results from missing historical predictions.

Code availability. Historical code is public at https://github.com/SoroushVahidi/combinatorial-opt-agent. The local M5 candidate adds deterministic verification entry points, manifests, source hashes and corrected evaluators. Its file-level inventory records included artifacts, sanitized derivatives, original licenses and concrete exclusions; preparation does not imply public availability of this revision.

Reproducibility. This revision uses frozen records and adds no model inference, solver execution or performance experiment. Released query metrics support aggregate checks; archived formulation logs support conditional objective comparisons; archived relationship responses support independent structure/agreement checks. Missing original headline scalar mappings, subset manifests, some rendered prompts and full-precision PaMOP returns prevent complete historical replay. Provider serving defaults and weight bytes are not fully authenticated. Compiling the source package reproduces the document, not missing measurements.

Declaration of generative AI and AI-assisted technologies in the writing process. During preparation the author used ChatGPT and Gemini to improve readability and language. OpenAI Codex later assisted manuscript revisions, evidence organization and deterministic validation. The author reviewed and edited the original work and remains responsible for its content; the M5 candidate awaits independent author review.

Declaration of AI assistance in software development. The author used Claude, Cursor and GitHub Copilot for coding assistance and reviewed, verified, tested and edited the resulting code. OpenAI Codex later assisted code corrections and offline verification scripts. The author retains responsibility for the implementation and reported results. Scientific LLM calls are identified separately; the evaluated scalar-grounding procedure uses no generative LLM inference.

References

[1] Affolter K, Stockinger K, Bernstein A (2019) A comparative survey of recent natural language interfaces for databases. The VLDB Journal 28(5):793–819. https://doi.org/10.1007/s00778-019-00567-8

[2] AhmadiTeshnizi A, Gao W, Udell M (2024) NLP4LP: A dataset for natural language to linear programming and mixed-integer linear programming. Hugging Face dataset repository, URL https://huggingface.co/datasets/udell-lab/NLP4LP, released with OptiMUS (ICML 2024); gated dataset

[3] AhmadiTeshnizi A, Gao W, Udell M (2024) OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In: Proceedings of the 41st International Conference on Machine Learning, Proceedings of Machine Learning Research, vol 235. PMLR, pp 577–596, URL https://proceedings.mlr.press/v235/ahmaditeshnizi24a.html

[4] Ahmed T, Choudhury S (2025) OPT2CODE: A retrieval-augmented framework for solving linear programming problems. Natural Language Processing Journal 13:100185. https://doi.org/10.1016/j.nlp.2025.100185, URL https://www.sciencedirect.com/science/article/pii/S2949719125000615

[5] Almasian S, Kazakova V, Göldner P, et al (2023) CQE: A comprehensive quantity extractor. In: Proceedings of EMNLP, pp 12845–12859, https://doi.org/10.18653/v1/2023.emnlp-main.793, URL https://aclanthology.org/2023.emnlp-main.793/

[6] Astorga N, Liu T, Xiao Y, et al (2025) Autoformulation of mathematical optimization models using LLMs. In: Proceedings of the 42nd International Conference on Machine Learning (ICML 2025), Proceedings of Machine Learning Research, vol 267. PMLR, pp 1864–1886, URL https://proceedings.mlr.press/v267/astorga25a.html

[7] Chen J, Xiao S, Zhang P, et al (2024) M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In: Findings of the Association for Computational Linguistics: ACL 2024. Association for Computational Linguistics, pp 2318–2335, https://doi.org/10.18653/v1/2024.findings-acl.137

[8] Chen Y, Xia J, Shao S, et al (2025) Solver-informed RL: Grounding large language models for authentic optimization modeling. In: Advances in Neural Information Processing Systems (NeurIPS), https://doi.org/10.52202/085713-3539, URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/98555b92e5dedbdad5921049281995bc-Abstract-Conference.html

[9] Deerwester S, Dumais ST, Furnas GW, et al (1990) Indexing by latent semantic analysis. Journal of the American Society for Information Science 41(6):391–407. https://doi.org/10.1002/(SICI)1097-4571(199009)41:6⟨391::

AID-ASI1⟩3.0.CO;2-9

[10] Deng X, Hassan Awadallah A, Meek C, et al (2021) Structure-grounded pretraining for text-to-SQL. In: Proceedings of NAACL-HLT, pp 1337–1350, https://doi.org/10.18653/v1/2021.naacl-main.105, URL https://aclanthology.org/2021.naacl-main.105/

[11] Ding Z, Tan Z, Zhang J, et al (2026) OR-R1: Automating modeling and solving of operations research optimization problem via test-time reinforcement learning. In: Proceedings of the Fortieth AAAI Conference on Artificial Intelligence (AAAI-26), pp 228–236, https://doi.org/10.1609/aaai.v40i1.36983

[12] Gao Z, Ge A, Berenbeim A, et al (2026) Models can model, but can't bind: Structured grounding in text-to-optimization. In: Conference on Language Modeling (COLM), URL https://arxiv.org/abs/2605.21751v2, version 2, 15 July 2026; accepted to COLM 2026

[13] Holm S (1979) A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics 6(2):65–70. URL https://www.jstor.org/stable/4615733

[14] Huang C, Tang Z, Hu S, et al (2025) ORLM: A customizable framework in training large models for automated optimization modeling. Operations Research 73(6):2986–3009. https://doi.org/10.1287/opre.2024.1233

[15] Jiang C, Shu X, Qian H, et al (2025) LLMOPT: Learning to define and solve general optimization problems from scratch. In: Proceedings of the Thirteenth International Conference on Learning Representations, URL https://proceedings.iclr.cc/paper_files/paper/2025/file/fbe6dd68b0cf2b1d43b458d2b8ca31b0-Paper-Conference.pdf

[16] Kadioglu S, Dakle PP, Uppuluri K, et al (2024) Ner4Opt: Named entity recognition for optimization modelling from natural language. Constraints 29(3–4):261–299. https://doi.org/10.1007/s10601-024-09376-5

[17] Kong M, Qu A, Guo X, et al (2025) AlphaOPT: Formulating optimization programs with self-improving LLM experience library. arXiv preprint arXiv:251018428 URL https://arxiv.org/abs/2510.18428, arXiv:2510.18428 [cs.AI]

[18] Kuhn HW (1955) The hungarian method for the assignment problem. Naval Research Logistics Quarterly 2(1–2):83–97. https://doi.org/10.1002/nav.3800020109, URL https://onlinelibrary.wiley.com/doi/10.1002/nav.3800020109

[19] Lhoest Q, Villanova del Moral A, Jernite Y, et al (2021) Datasets: A community library for natural language processing. In: Proceedings of the 2021 Conference

on Empirical Methods in Natural Language Processing: System Demonstrations, pp 175–184, https://doi.org/10.18653/v1/2021.emnlp-demo.21, URL https://aclanthology.org/2021.emnlp-demo.21/

[20] Li Y, Wang H, Xue B, et al (2026) Solver-independent automated problem formulation via LLMs for high-cost simulation-driven design. In: Findings of the Association for Computational Linguistics: ACL 2026. Association for Computational Linguistics, https://doi.org/10.18653/v1/2026.findings-acl.102

[21] Li Z, Lu H, Wei T, et al (2026) Constructing industrial-scale optimization modeling benchmark. arXiv preprint arXiv:260210450 https://doi.org/10.48550/arXiv.2602.10450, URL https://arxiv.org/abs/2602.10450

[22] Lu H, Xie Z, Wu Y, et al (2025) OptMATH: A scalable bidirectional data synthesis framework for optimization modeling. In: Proceedings of the 42nd International Conference on Machine Learning (ICML 2025), Proceedings of Machine Learning Research, vol 267. PMLR, pp 40769–40802, URL https://proceedings.mlr.press/v267/lu25o.html

[23] Pan X, Fang J, Wu F, et al (2025) Guiding large language models in modeling optimization problems via question partitioning. In: Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI-25), pp 2657–2665, https://doi.org/10.24963/ijcai.2025/296, URL https://www.ijcai.org/proceedings/2025/296

[24] Pedregosa F, Varoquaux G, Gramfort A, et al (2011) Scikit-learn: Machine learning in Python. Journal of Machine Learning Research 12:2825–2830

[25] Punyakanok V, Roth D, Yih Wt, et al (2004) Semantic role labeling via integer linear programming inference. In: Proceedings of COLING, pp 1346–1352, URL https://aclanthology.org/C04-1197/

[26] Ramamonjison R, Li H, Yu TTL, et al (2022) Augmenting operations research with auto-formulation of optimization models from problem descriptions. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track. Association for Computational Linguistics, pp 29–62, https://doi.org/10.18653/v1/2022.emnlp-industry.4, URL https://aclanthology.org/2022.emnlp-industry.4/

[27] Ramamonjison R, Yu T, Li R, et al (2022) NL4Opt competition: Formulating optimization problems based on their natural language descriptions. In: Proceedings of the NeurIPS 2022 Competitions Track, Proceedings of Machine Learning Research, vol 220. PMLR, pp 189–203, URL https://proceedings.mlr.press/v220/ramamonjison23a.html, publisher citation year 2022; proceedings PDF masthead 2023

[28] Robertson S, Zaragoza H (2009) The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval 3(4):333–389. https://doi.org/10.1561/1500000019

[29] Roy S, Roth D (2017) Unit dependency graph and its application to arithmetic word problem solving. In: Proceedings of the AAAI Conference on Artificial Intelligence, https://doi.org/10.1609/aaai.v31i1.10959, URL https://ojs.aaai.org/index.php/AAAI/article/view/10959

[30] Scholak T, Schucher N, Bahdanau D (2021) PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, pp 9895–9901, https://doi.org/10.18653/v1/2021.emnlp-main.779, URL https://aclanthology.org/2021.emnlp-main.779/

[31] Sheetrit E, Brief M, Mishaeli M, et al (2024) ReMatch: Retrieval enhanced schema matching with LLMs. arXiv preprint arXiv:240301567 https://doi.org/10.48550/arXiv.2403.01567, URL https://arxiv.org/abs/2403.01567

[32] Thind R, Sun Y, Liang L, et al (2025) OptimAI: Optimization from natural language using LLM-powered AI agents. arXiv preprint arXiv:250416918 URL https://arxiv.org/abs/2504.16918, arXiv:2504.16918 [cs.CL]

[33] Wang B, Shin R, Liu X, et al (2020) RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics, pp 7567–7578, https://doi.org/10.18653/v1/2020.acl-main.677, URL https://aclanthology.org/2020.acl-main.677/

[34] Wang C, Tatwawadi K, Brockschmidt M, et al (2018) Robust text-to-sql generation with execution-guided decoding. arXiv preprint arXiv:180703100 https://doi.org/10.48550/arXiv.1807.03100, URL https://arxiv.org/abs/1807.03100

[35] Xiao Z, Zhang D, Wu Y, et al (2024) Chain-of-experts: When LLMs meet complex operations research problems. In: Proceedings of the Twelfth International Conference on Learning Representations, URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/d45ee77826332c100a1e15f7765b99ff-Abstract-Conference.html

[36] Xiao Z, Wang YJ, Han X, et al (2026) DeepOR: A deep reasoning foundation model for optimization modeling. In: Proceedings of the Fortieth AAAI Conference on Artificial Intelligence (AAAI-26), pp 34052–34060, https://doi.org/10.1609/aaai.v40i40.40699

[37] Xie W, Chen Y, Hu YX, et al (2026) Escaping the sisyphus dilemma: Experience replay for robust text-to-optimization modeling. In: Findings of the Association for Computational Linguistics: ACL 2026. Association for Computational

Linguistics, https://doi.org/10.18653/v1/2026.findings-acl.690

[38] Yang B, Zhou Q, Li J, et al (2026) ORThought: Benchmarking and automating logistics optimization modeling via structured LLM reasoning. Artificial Intelligence for Transportation 6:100059. https://doi.org/10.1016/j.ait.2026.100059, URL https://doi.org/10.1016/j.ait.2026.100059

[39] Yu L, Zhao X, Li D, et al (2026) Uncertainty-aware test-time search for optimization problem solving. In: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, https://doi.org/10.18653/v1/2026.acl-long.1975

[40] Yu T, Zhang R, Yang K, et al (2018) Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, pp 3911–3921, https://doi.org/10.18653/v1/D18-1425, URL https://aclanthology.org/D18-1425/

[41] Zhai H, Lawless C, Vitercik E, et al (2025) EquivaMap: Leveraging LLMs for automatic equivalence checking of optimization formulations. In: Proceedings of the 42nd International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, vol 267. PMLR, pp 74288–74305, URL https://proceedings.mlr.press/v267/zhai25a.html

[42] Zhang J, Wang W, Guo S, et al (2024) Solving general natural-language-description optimization problems with large language models. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track). Association for Computational Linguistics, pp 483–490, https://doi.org/10.18653/v1/2024.naacl-industry.42, URL https://aclanthology.org/2024.naacl-industry.42/

[43] Zhang X, Wan Y, Zhang B, et al (2026) Dual-cluster memory agent: Resolving multi-paradigm ambiguity in optimization problem solving. In: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, https://doi.org/10.18653/v1/2026.acl-long.266